Complete Technical & Operational Specification: Z Marketplace Platform

1. System Overview & Core Operating Philosophy

Z is a hyper-localized, multi-vendor Software-as-a-Service (SaaS) and e-commerce marketplace built specifically for the Namibian market. The platform bridges individual merchants and registered retail businesses across the country with local consumers.

The Central Escrow Model

The platform operates on **The Central Escrow Model**.

- **Payment Settle:** All customer checkout funds across Namibia settle instantly into Z's central corporate account via the payment gateway.
- **Platform Role:** Z acts as a trusted escrow intermediary. The platform does not deploy real-time automated payouts via third-party banking APIs. Instead, it utilizes a robust internal database ledger to capture, hold, and track distinct merchant balances.
- **Distributions:** Funds are manually distributed by the platform operator to merchants via local electronic fund transfers (EFT) during an isolated weekly administrative window.

---

2. Master Database & Collection Architecture (MongoDB / Mongoose)

The platform leverages an **Embedded-Hybrid relational document model** inside MongoDB. This approach encapsulates complex multi-vendor transactions cleanly within single parent records while isolating merchant logistics fulfillment states.

Primary Collections Layout

A. Users Collection (Buyers & Sellers Unified)

javascript

```
const UserSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  phoneMobile: { type: String, required: true }, // Format: +26481...
  role: { type: String, enum: ['BUYER', 'SELLER', 'ADMIN'], default: 'BUYER' },

  // Verification State Machine for Sellers
```

```
  sellerVerification: {
    status: { type: String, enum: ['NONE', 'PENDING_VERIFICATION', 'VERIFIED', 'SUSPENDED'], default: 'NONE' },
    entityType: { type: String, enum: ['INDIVIDUAL', 'REGISTERED_BUSINESS'] },
    idDocumentUrl: { type: String }, // Secure S3 link to Government ID / BIPA Certificate
    submittedAt: Date,
    verifiedAt: Date
  },

  // Linked Bank details for manual EFT processing
  bankAccount: {
    bankName: { type: String, enum: ['FNB', 'Standard Bank', 'Bank Windhoek', 'Nedbank'] },
    accountName: { type: String },
    accountNumber: { type: String },
    branchCode: { type: String },
    isVerified: { type: Boolean, default: false }
  },

  // Internal Ledger Balance Tracker
  wallet: {
    pendingBalanceNAD: { type: Number, default: 0 },     // Funds held securely in escrow
    withdrawableBalanceNAD: { type: Number, default: 0 }  // Funds cleared post-cooling window
  },

  isSoftDeleted: { type: Boolean, default: false },
  createdAt: { type: Date, default: Date.now }
});
```

Use code with caution.

B. Products Collection

javascript

```
const ProductSchema = new mongoose.Schema({
  sellerId: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  name: { type: String, required: true, text: true }, // Text index initialized for basic search
  description: { type: String, required: true, text: true },
  priceNAD: { type: Number, required: true },
  stock: { type: Number, required: true, min: 0 },
  images: [{ type: String }], // Array of sanitized secure Object Storage URLs
  isAvailable: { type: Boolean, default: true },
  hiddenByModerator: { type: Boolean, default: false },
  reportCount: { type: Number, default: 0 },
  createdAt: { type: Date, default: Date.now }
});
```

Use code with caution.

C. Orders Collection (The Core Split Engine)

javascript

```
const OrderSchema = new mongoose.Schema({
  buyerId: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },

  payment: {
    transactionId: { type: String, required: true }, // DPO Provider reference trace
    status: { type: String, enum: ['PENDING', 'PAID', 'FAILED', 'REFUNDED'], default:
'PENDING' },
    totalAmountNAD: { type: Number, required: true }, // Global cart total captured
    paidAt: { type: Date }
  },
```

```
  // The Master Split Array allowing multi-seller checkouts
  subOrders: [{
    sellerId: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
    subOrderId: { type: String, required: true }, // Structured format: Z-YYYYMMDD-[SHORT_ID]

    items: [{
      productId: { type: mongoose.Schema.Types.ObjectId, ref: 'Product' },
      name: { type: String, required: true }, // Snapshot string preservation
      priceNAD: { type: Number, required: true },
      quantity: { type: Number, required: true }
    }],

    financials: {
      subTotal: { type: Number, required: true },
      shippingFee: { type: Number, required: true },
      commissionDeducted: { type: Number, required: true }, // Z's calculated cut
      finalSellerPayout: { type: Number, required: true }  // Balance due to merchant
    },

    fulfillment: {
      status: {
        type: String,
        enum: ['PROCESSING', 'SHIPPED', 'DELIVERED', 'COMPLETED', 'DISPUTED', 'REFUNDED'],
        default: 'PROCESSING'
      },
      method: { type: String, required: true }, // E.g., "Formula Courier", "Oshakati Taxi"
```

```
    trackingNumber: { type: String },
    updatedAt: { type: Date, default: Date.now },
    coolingPeriodExpiresAt: { type: Date } // Threshold date marking the release of escrow funds
  }
 }],

 createdAt: { type: Date, default: Date.now }
});
```

Use code with caution.

---

3. Financial Mechanics & Calculation Rules

The Split-Commission Formula

Commissions are calculated **exclusively on the item product subtotal** within each unique seller sub-order bucket. No platform fees are assessed against shipping line-items.

1. **Platform Fee Rate:** Hardcoded at 10% base commission.
2. **Platform Fee Calculation:** $\text{Platform Commission}=\text{Product Subtotal}\times 0.10$
3. **Vendor Credit Assignment:** $\text{Final Seller Payout}=(\text{Product Subtotal}+\text{Shipping Fee})-\text{Platform Commission}$
4. **Gateway Costs:** Payment processing transactional cuts levied by DPO Group (approx. 3.5% + N$2.00) are absorbed directly by Z out of its 10% platform revenue slice, guaranteeing structural pricing clarity for local merchants.
5. **Decimal Precision Guardrail:** Every database financial field calculations run inside integer conversions or strict format filters (.toFixed(2)) to eliminate JavaScript floating-point rounding bugs.

Version-1 Merchant Logistics Parameters

Sellers set their own shipping charges at the profile dashboard level using two structural layers:

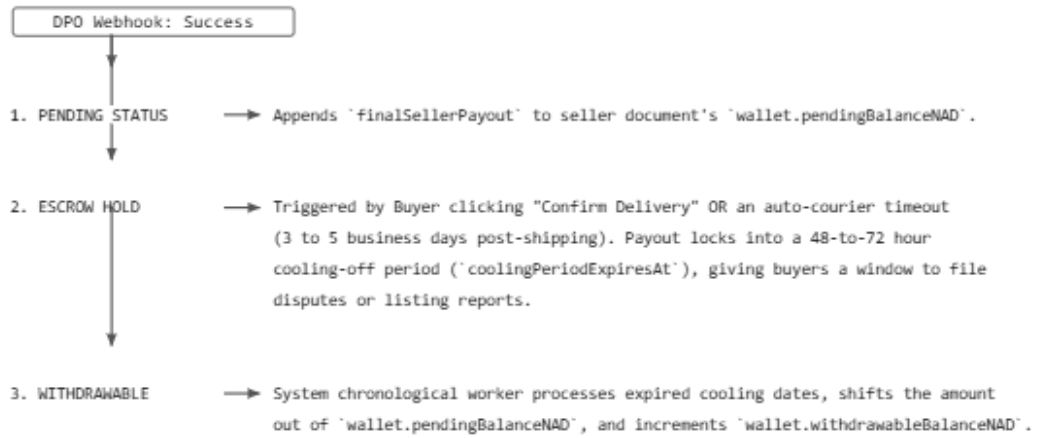
- **Tiered Flat-Rate Per Major Region:**
    - *Local Town Tier:* (e.g., N$30 flat) applied when buyer destination matches seller hub town.
    - *Main Hub Inter-Town Tier:* (e.g., N$60 - N$80 flat) targeting transit routes via national couriers between towns like Windhoek, Swakopmund, Walvis Bay, or Oshakati.
    - *Outlying Tier:* (e.g., N$120 flat) protecting vendors against remote route surcharges.
- **Free Shipping Threshold Integration:** Merchants can toggle an optional rule assigning free shipping (shippingFee = 0) if the cumulative line-item subtotal from *their shop alone* crosses a specific limit (e.g., spending N$800+).

---

4. Escrow State Machine & Manual EFT Pipeline

The Three-Stage Balance Flow

To completely safeguard Z from financial damage due to non-fulfillment or fraudulent items, merchant capital proceeds through three locked database transitions:



Show Code

Download diagram SVG

The Reverse-Ledger Refund Engine

If an item is damaged or never sent, your database handles a strict rollback process:

- The application runs an online transaction reversal call back to the DPO API, moving the money back to the consumer's card cardholder bank.
- The platform updates the database document status to REFUNDED and executes a **negative balance adjustment query** against the merchant's wallet.pendingBalanceNAD, reclaiming the balance before any funds can clear.

Weekly Manual EFT Execution Protocol

- **Schedule Window:** Performed once weekly (e.g., "Payout Fridays").
- **System Action:** The admin panel outputs an automated query list isolating all merchants where wallet.withdrawableBalanceNAD >= 100 alongside their verified bank details (bankAccount).
- **Human Checkpoint:** The platform operator reviews the export list for outstanding dispute logs, logs into Z's commercial business banking portal, and runs the batch manual transfers safely. The operator then posts a tracking confirmation ledger update to empty the seller's active withdrawable balance container back to zero.

---

5. Pre-Launch Security & Technical Infrastructure Hardening

API and Route Protections

- **Atomic Inventory Reservation:** To eliminate race conditions where multiple active browser instances attempt to buy identical stock volumes simultaneously, checkouts employ a conditional collection query ($inc matched with a $gte: quantity validation filter). Stock blocks drop for a temporary 15-minute window during active card submission routes. If payment fails, the system rolls back product quantities automatically.
- **Rate Limiting Interceptors:** Mandatory middleware rules block high-frequency bots on /login, /register, and /forgot-password routes, limiting inputs to 5 hits per 15-minute block per IP address.

Media & Media Storage Sanitization

- **The Pipeline:** Merchants never upload product photos directly to the web app server. The client requests a **Signed Upload URL** via a verified backend authentication handshake. The browser then transmits assets directly to an isolated Cloud Object Storage infrastructure container (e.g., Amazon S3 or DigitalOcean Spaces).

- **Sanitization Filters:** Upload rules enforce strict 5MB size limits, strip geographic metadata (EXIF tracking tags) automatically to ensure vendor privacy, and filter content extensions to valid MIME-types (image/jpeg, image/png, image/webp).

Platform Operational Moderation

- **Crowdsourced Flagging:** Every item storefront layout features a "Report Listing" element. If a single product gathers a specific volume of unique reports (e.g., 3 flags), the backend triggers a hiddenByModerator = true statement, removing the item from category indexing queues until manual staff review occurs.
- **Compliance Compliance (Soft-Deletion Protocol):** Account removal requests do not use hard delete queries. Financial records must remain discoverable for historical audit loops and NamRA tax validations. The platform initiates a soft-delete toggle (isSoftDeleted = true), masking name, password, email, and phone arrays while preserving transactional data securely.

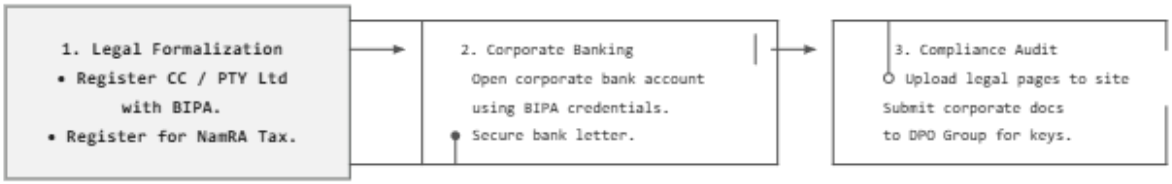
Communication & Monitoring Systems

- **Mobile Notification Dispatchers:** The platform loop utilizes a third-party connection gateway (e.g., Twilio API) to bypass email bottlenecks. Successful payment hooks automatically send instant SMS or WhatsApp alerts to each individual vendor involved in a mixed-seller checkout transaction, alerting them to check their dashboard and prepare their parcels.
- **Uptime Observers:** External automated ping scripts (e.g., UptimeRobot) continuously inspect the platform routes every 60 seconds, sending immediate alerts to the technical team if database locks or runtime exceptions cause platform downtime.

---

6. Pre-Launch Legal & Merchant Account Roadmap

To transition your system from sandbox developer mock testing to a live, payment-enabled platform processing real Namibian Dollars, you must complete the following real-world milestones:



Show Code

Download diagram SVG

Your system prototype is architecturally set up to transition into a powerful, scalable software asset. Use this master document to code, protect, and test your platform components systematically inside your developer workspace.

Now that your entire master specification is fully laid out and documented, what **specific development milestone** or module would you like to start building first?